Zoe Protocol Lab

Zoe / allgems

Explore the guide

0.1 Open and export 2
0.2 SPECS framing 2
0.3 The 128/129 boundary 2
0.4 AMAC replies and read-next 3
0.5 Evidence boundaries 3
0.6 Verification 3

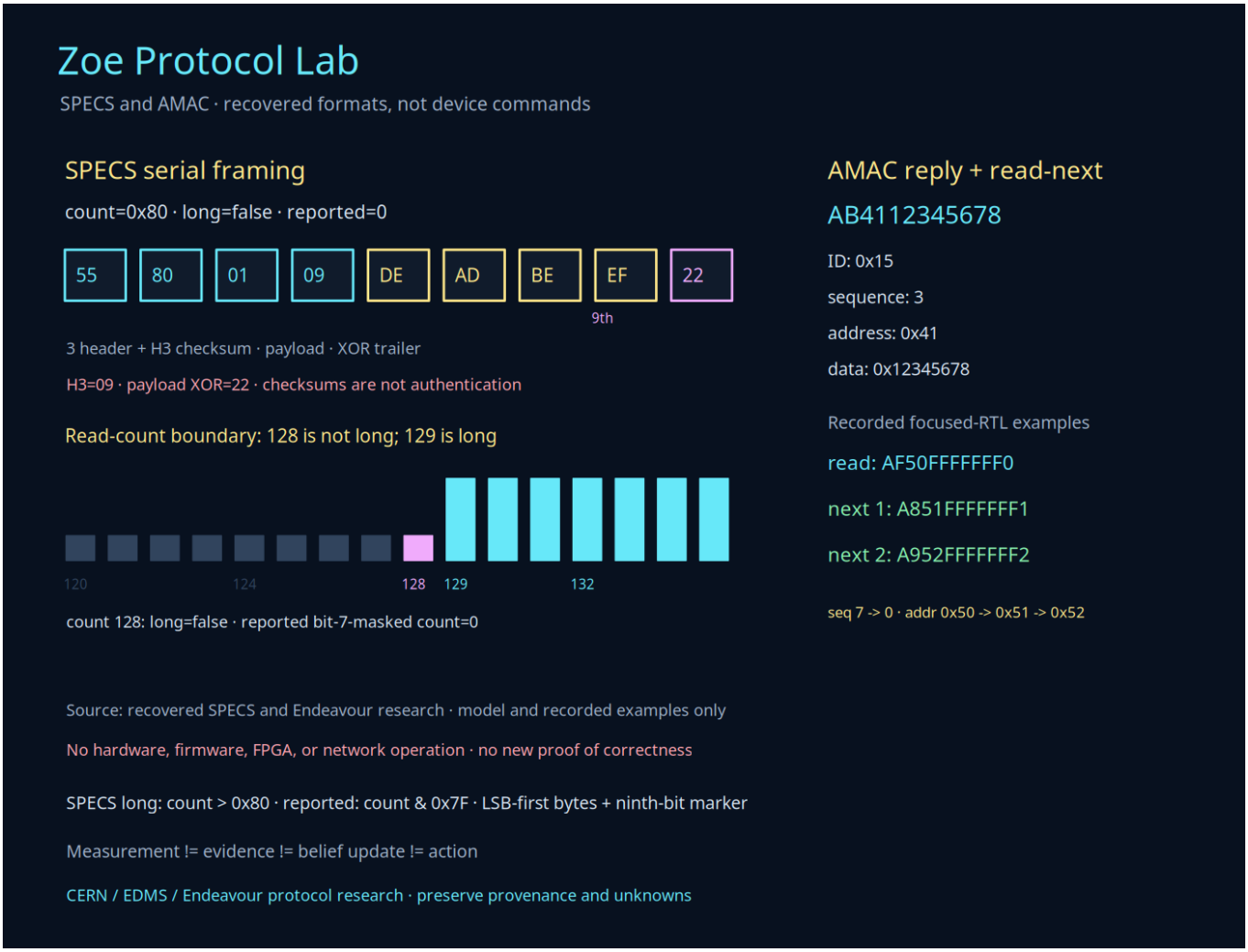


Figure 1: Zoe Protocol Lab · visual study

The Protocol Lab turns recovered SPECS and AMAC behavior into a small, inspectable native instrument.

It provides real codecs and a visual explanation rather than another anonymous waveform viewer.

No hardware, firmware, FPGA, or network operation is performed. The examples come from documented analysis and focused RTL simulations, not new proof of correctness, timing closure, or deployed-system acceptance.

0.1 Open and export

```
zoe protocol
zoe protocol --count 129
zoe protocol --count 0x80 --output protocol.svg
zoe protocol --output protocol.gem
```

The same options work through `gem protocol`. The native selector changes the read count from 0 to 255 and exports the current scene as SVG, PNG, or GEM VDI. Output files are never overwritten.

0.2 SPECS framing

The first header byte is `(slave_address << 2) | mode`. The address is six bits and modes are I2C, JTAG, parallel bus, and register. For example:

- `0x15 << 2 | i2c = 0x54`.
- `0x15 << 2 | jtag = 0x55`.

The demonstration preserves the documented frame:

```
55 02 01 03 DE AD BE EF 22
```

It contains three header bytes, a header-checksum byte, four payload bytes, and a payload-checksum byte.

The header check is:

```
C = XOR(i=0..2) ((H_i >> 4) ^ (H_i & 0x0F)) ^ (H3 >> 4)
```

The receiver requires `H3 & 0x0F == C`. With a zero high nibble in H3, this example produces `03`. The trailer is the XOR of all payload bytes, producing `22` in this example. These checks provide integrity information, **not cryptographic authentication**.

Payload bytes are sent least-significant-bit first. Eight data clocks are followed by a ninth bit that is set only on the final payload byte. The ninth bit is not set on the trailer. Encoding and decoding validate the checksum and marker independently.

0.3 The 128/129 boundary

The recovered implementation uses:

```
long = read_count > 0x80
reported_count = read_count & 0x7F
```

Therefore:

- 128 is **not long** and reports 0.
- 129 is **long** and reports 1.
- 255 is **long** and reports 127.

The focused RTL evidence covers all 256 read counts and 1,280 checks. This is not the same as an informal comment suggesting that 255 is the only long count. The implementation's behavior takes precedence over that comment.

The lab's control regenerates the frame header and checksum for the selected count without generating a device command.

0.4 AMAC replies and read-next

A read reply has an eight-bit status/identity byte, an eight-bit address, and four data bytes in big-endian order:

```
AB 41 12 34 56 78
```

The decoder recovers:

- communications ID `0x15` from the upper five bits;
- sequence `3` from the lower three bits;
- address `0x41`;
- data `0x12345678`.

The focused RTL examples show:

Operation	Reply
Read address `0x50`	`AF50FFFFFFF0`
First read-next	`A851FFFFFFF1`
Second read-next	`A952FFFFFFF2`

Read-next increments address and sequence independently of the returned data. The three-bit sequence wraps from 7 to 0, and the eight-bit address wraps from 255 to 0. The demonstration's different register values are supplied separately; read-next does not itself increment the data.

0.5 Evidence boundaries

The local CERN reports preserve original hashes, recovered code semantics, dynamic RTL results, and explicit limits. The Protocol Lab implements the documented model independently; it does not copy the external WDB reader, execute proprietary HDL, or claim their hardware results.

The example output is a model and a replay of recorded examples. It is not a new dynamic simulation, a new measurement, or evidence that a deployed device will accept the same frames.

0.6 Verification

```
PYTHONPATH=guiai-lab python -m unittest stress.protocol_stress -v
```

Tests cover the exact frame, checksum and ninth-bit failures, empty payloads, invalid fields, all 256 count values, AMAC reply decoding, address and sequence wraparound, geometry, VDI, no-overwrite exports, and actual native count changes.